

Τμήμα Μηχανικών Η/Υ και Πληροφορικής

Παράλληλη Επεξεργασία

ParallelProcessing_Project_2026

Επώνυμο	Όνομα	ΑΜ
Θεοδοσόπουλος	Χριστόφορος	1100557

1. Εισαγωγή

Στην παρούσα εργασία υλοποιήθηκε σύστημα παράλληλης κανονικοποίησης για πίνακες τύπου double διαστάσεων N × D, αποθηκευμένους σε raw binary αρχεία χωρίς επικεφαλίδα. Υποστηρίζονται δύο μετασχηματισμοί:
StandardScaler ($x' = (x - \mu) / \sigma$) και
MinMaxScaler ($x' = (x - \min) / (\max - \min)$).

Η επεξεργασία ακολουθεί λογική δύο φάσεων: στην πρώτη φάση διαβάζονται τα δεδομένα σε blocks και υπολογίζονται τα στατιστικά ανά στήλη (μέσος όρος, τυπική απόκλιση, ελάχιστο, μέγιστο), και στη δεύτερη φάση εφαρμόζεται ο μετασχηματισμός και το αποτέλεσμα γράφεται σε νέο αρχείο. Η block-based επεξεργασία διασφαλίζει ότι η εφαρμογή λειτουργεί ακόμα και όταν το dataset υπερβαίνει τη διαθέσιμη μνήμη (out-of-core εκτέλεση).

Υλοποιήθηκαν πέντε εκδόσεις, καθεμία αξιοποιώντας διαφορετικό μηχανισμό παραλληλισμού:

- Σειριακή (αναφοράς): βάση ορθότητας και υπολογισμού speedup
- SIMD/AVX2: παραλληλισμός σε επίπεδο εντολών, με επεξεργασία 4 τιμών double ταυτόχρονα
- OpenMP: πολυνηματική επεξεργασία σε σύστημα κοινής μνήμης
- MPI: κατανεμημένη επεξεργασία με παράλληλο I/O μέσω MPI I/O
- CUDA: επιτάχυνση υπολογισμών μέσω GPU

2.Αναλυτική τεκμηρίωση παράλληλων υλοποιήσεων και τυχόν βελτιστοποιήσεων

4.1. Σειριακή Υλοποίηση Αναφοράς (Serial Implementation Reference)

Στο ερώτημα 4.1 υλοποιήθηκε η σειριακή έκδοση του αλγορίθμου feature scaling σε γλώσσα C.Ο αλγόριθμος εκτελείται σε δύο βασικές φάσεις.

- Στην **πρώτη φάση** διαβάζεται το αρχείο εισόδου σε blocks γραμμών και υπολογίζονται τα στατιστικά στοιχεία κάθε χαρακτηριστικού, δηλαδή το άθροισμα, το άθροισμα τετραγώνων, η ελάχιστη τιμή και η μέγιστη τιμή. Από αυτά υπολογίζονται στη συνέχεια ο μέσος όρος, η διακύμανση και η τυπική απόκλιση κάθε στήλης.
- Στη **δεύτερη φάση** το αρχείο εισόδου διαβάζεται ξανά και εφαρμόζεται ο επιλεγμένος μετασχηματισμός scaling. Στην περίπτωση standard scaling εφαρμόζεται ο τύπος: x' = (x - mean[j]) / stddev[j] ,ενώ στην περίπτωση min-max scaling εφαρμόζεται ο τύπος: x' = (x - min[j]) / (max[j] - min[j])

Το αποτέλεσμα γράφεται σε νέο δυαδικό αρχείο εξόδου το out_serial.bin ή το out_serial_minmax.bin ανάλογα ποια έκδοση θέλουμε να τρέξουμε.Η ανάγνωση του αρχείου γίνεται από την εντολή: FILE* fin = fopen(input_file, "rb");
Η ανάγνωση γίνεται σε blocks με:
size_t elements_read = fread(buffer, sizeof(double), elements_to_read, fin);
Αυτό διαβάζει από το αρχείο elements_to_read στοιχεία, όπου κάθε στοιχείο έχει μέγεθος sizeof(double), και τα αποθηκεύει στον πίνακα buffer.
Για την **επεξεργασία μεγάλων αρχείων με blocks**, χρησιμοποιείται αυτό το κομμάτι:
const size_t BLOCK_ROWS = 100000;
double* buffer = (double*)malloc(BLOCK_ROWS * D * sizeof(double));

Για τον **υπολογισμό στατιστικών ανά στήλη**, χρησιμοποιούνται οι πίνακες:
double* sum = (double*)calloc(D, sizeof(double));
double* sumsq = (double*)calloc(D, sizeof(double));
double* minv = (double*)malloc(D * sizeof(double));
double* maxv = (double*)malloc(D * sizeof(double));

Ο αλγόριθμος εκτελείται σε δύο φάσεις. Στην πρώτη φάση διαβάζεται το αρχείο εισόδου και υπολογίζονται στατιστικά στοιχεία ανά στήλη. Για κάθε χαρακτηριστικό υπολογίζονται το άθροισμα των τιμών, το άθροισμα των τετραγώνων, η ελάχιστη τιμή και η μέγιστη τιμή. Με βάση αυτά υπολογίζονται στη συνέχεια ο μέσος όρος, η διακύμανση και η τυπική απόκλιση κάθε στήλης.

Ο μέσος όρος κάθε στήλης υπολογίζεται ως:

mean[j] = sum[j] / N

Η διακύμανση υπολογίζεται ως:

variance[j] = sumsq[j] / N - mean[j]^2

και η τυπική απόκλιση ως:

stddev[j] = sqrt(variance[j])

Στη δεύτερη φάση το αρχείο εισόδου ανοίγεται ξανά και εφαρμόζεται ο επιλεγμένος μετασχηματισμός scaling. Αν έχει επιλεγεί το StandardScaler, κάθε τιμή μετατρέπεται σύμφωνα με τον τύπο:

x' = (x - mean[j]) / stddev[j]

όπου j είναι η στήλη στην οποία ανήκει η τιμή. Με αυτόν τον τρόπο κάθε χαρακτηριστικό αποκτά μέση τιμή περίπου 0 και τυπική απόκλιση περίπου 1.

Αν έχει επιλεγεί το MinMaxScaler, κάθε τιμή μετατρέπεται σύμφωνα με τον τύπο:

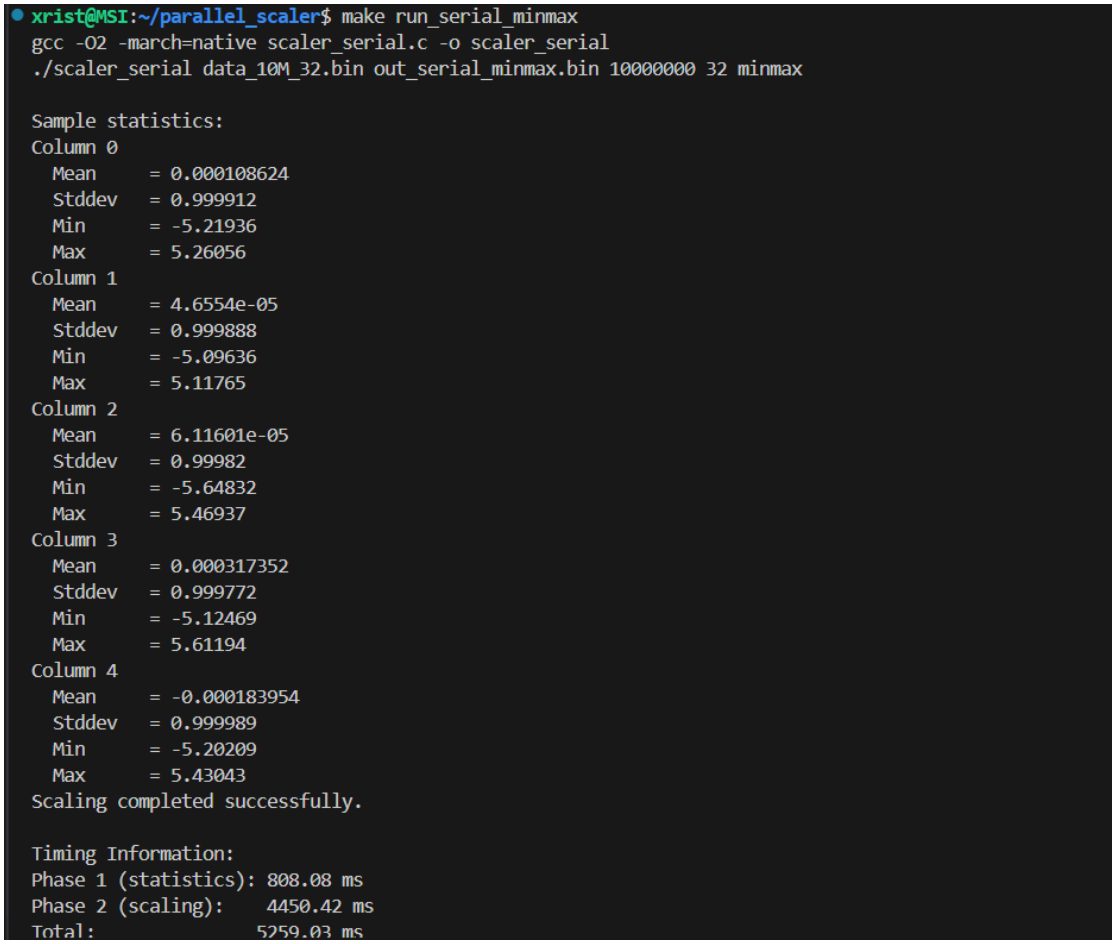
x' = (x - min[j]) / (max[j] - min[j])

Με αυτόν τον μετασχηματισμό οι τιμές κάθε χαρακτηριστικού κανονικοποιούνται στο διάστημα [0, 1]. Στην περίπτωση όπου η διαφορά max[j] - min[j] είναι πολύ μικρή, χρησιμοποιείται παρονομαστής ίσος με 1, ώστε να αποφευχθεί διαίρεση με μηδέν.

Μετά την εφαρμογή του scaling σε κάθε block, τα κανονικοποιημένα δεδομένα γράφονται σε νέο raw binary αρχείο εξόδου. Το αρχείο εξόδου έχει την ίδια διάταξη με το αρχείο εισόδου, δηλαδή περιέχει συνεχόμενες τιμές double σε μορφή N x D.

Συνολικά, η σειριακή υλοποίηση αποτελεί τη βασική έκδοση του scaler. Υποστηρίζει ανάγνωση raw binary αρχείων, υπολογισμό στατιστικών ανά στήλη, εφαρμογή StandardScaler και MinMaxScaler, εγγραφή raw binary αρχείου εξόδου και επεξεργασία μεγάλων αρχείων με χρήση blocks.

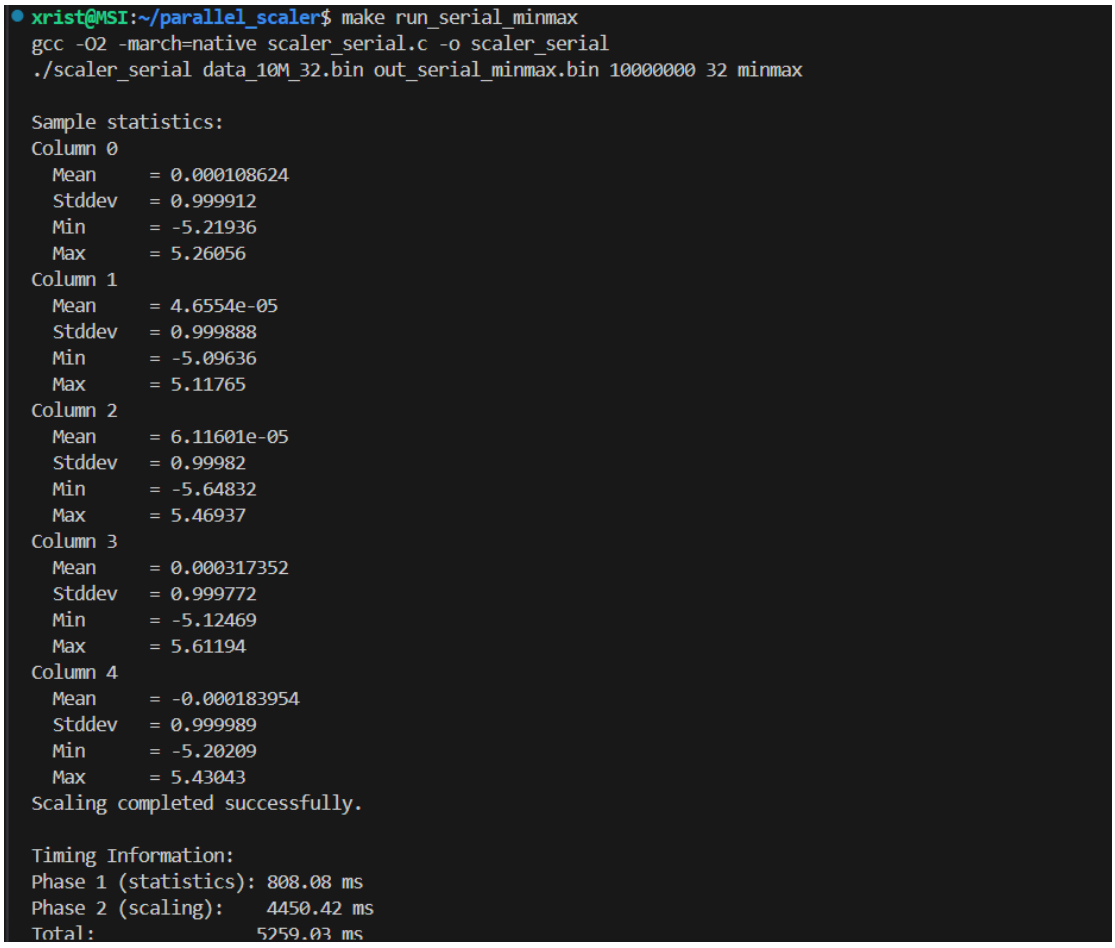
Screenshot εκτέλεσης scaler_serial.c με την μέθοδο standard:



Από την έξοδο της σειριακής εκτέλεσης παρατηρούμε ότι για τις πρώτες στήλες ο μέσος όρος είναι πολύ κοντά στο 0, ενώ η τυπική απόκλιση είναι πολύ κοντά στο 1. Αυτό δείχνει ότι τα αρχικά δεδομένα έχουν ήδη περίπου κανονική κατανομή και είναι σχεδόν standardized. Παρόλα αυτά, το πρόγραμμα εφαρμόζει κανονικά το standard scaling, ώστε το τελικό αρχείο εξόδου να περιέχει τιμές με μέσο όρο περίπου 0 και τυπική απόκλιση περίπου 1 για κάθε χαρακτηριστικό.

Οι τιμές min και max βρίσκονται περίπου στο διάστημα [-5, 5], κάτι που είναι λογικό για μεγάλο πλήθος δειγμάτων που ακολουθούν κανονική κατανομή.

Screenshot εκτέλεσης scaler_serial.c με την μέθοδο minmax:



```
xrist@MSI:~/parallel_scaler$ make run_serial_minmax
gcc -O2 -march=native scaler_serial.c -o scaler_serial
./scaler_serial data_10M_32.bin out_serial_minmax.bin 10000000 32 minmax

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information:
Phase 1 (statistics): 808.08 ms
Phase 2 (scaling):    4450.42 ms
Total:                5259.03 ms
```

4.2 SIMD Υλοποίηση

Στη SIMD υλοποίηση υλοποιήθηκε η ίδια λειτουργία με τη σειριακή έκδοση, δηλαδή feature scaling σε δύο φάσεις, αλλά με χρήση AVX2 SIMD εντολών. Το πρόγραμμα υποστηρίζει δύο modes εκτέλεσης: standard scaling και min-max scaling.

Σε αυτή τη φάση επιχειρήθηκε:
1.**Vectorization**: κυρίως στον υπολογισμό των `sum` και `sumsq`. Συγκεκριμένα, με AVX2 φορτώνονται 4 τιμές τύπου double ταυτόχρονα και ενημερώνονται τα αθροίσματα με SIMD πράξεις. Για το `sumsq` χρησιμοποιείται η εντολή fused multiply-add, ώστε να υπολογίζεται το `x * x + sumsq` με μία διανυσματική πράξη.

Οι υπολογισμοί `min` και `max` γίνονται κυρίως scalar μέσα στο SIMD loop. Αυτό σημαίνει ότι η πρώτη φάση δεν είναι πλήρως vectorized, αλλά μόνο ένα μέρος της, συγκεκριμένα τα αθροίσματα και τα αθροίσματα τετραγώνων.
Στη δεύτερη φάση γίνεται η εφαρμογή του scaling στα δεδομένα. Εδώ η SIMD υλοποίηση είναι πιο άμεση, επειδή κάθε στοιχείο μπορεί να μετασχηματιστεί ανεξάρτητα.
2.Η SIMD υλοποίηση έγινε με **explicit intrinsics AVX2**, δηλαδή με εντολές όπως:`_mm256_loadu_pd`, `_mm256_storeu_pd`, `_mm256_add_pd`, `_mm256_sub_pd`, `_mm256_div_pd`, `_mm256_fmadd_pd`, `_mm256_cmp_pd` και `_mm256_blendv_pd`.
3.Η μεταγλώττιση της SIMD έκδοσης έγινε με τα εξής **compiler_flags:**
gcc -O3 -march=native -mavx2 -mfma scaler_simd.c -o scaler_simd

- Το `-O3` ενεργοποιεί υψηλό επίπεδο βελτιστοποιήσεων
- το `-march=native` επιτρέπει στον compiler να αξιοποιήσει τις δυνατότητες του συγκεκριμένου επεξεργαστή
- το `-mavx2` ενεργοποιεί τις AVX2 εντολές
- και το `-mfma` ενεργοποιεί τις fused multiply-add εντολές.

4. Η SIMD υλοποίηση στόχευε στη βελτίωση της απόδοσης μέσω της ταυτόχρονης επεξεργασίας 4 τιμών double με AVX2 εντολές. Θεωρητικά, αυτό μπορεί να μειώσει τον χρόνο των αριθμητικών πράξεων, επειδή αντί να επεξεργάζεται μία τιμή κάθε φορά, ο επεξεργαστής επεξεργάζεται τέσσερις τιμές με μία εντολή.

Στην πράξη, η βελτίωση της απόδοσης εξαρτάται από τους χρόνους εκτέλεσης που μετρήθηκαν. Αν ο συνολικός χρόνος της SIMD έκδοσης είναι μικρότερος από τον χρόνο της σειριακής έκδοσης, τότε η SIMD υλοποίηση πέτυχε επιτάχυνση.
Το speedup υπολογίζεται ως:

Speedup = T_serial / T_SIMD

Όπου `T_serial` είναι ο χρόνος της σειριακής έκδοσης και `T_SIMD` ο χρόνος της SIMD έκδοσης.
Στη συγκεκριμένη εκτέλεση η SIMD υλοποίηση δεν παρουσίασε σημαντική βελτίωση απόδοσης σε σχέση με τη σειριακή έκδοση. Παρότι χρησιμοποιούνται AVX2 intrinsics, ο συνολικός χρόνος επηρεάζεται έντονα από άλλους παράγοντες, όπως η ανάγνωση και εγγραφή μεγάλων αρχείων, οι προσβάσεις στη μνήμη και το γεγονός ότι δεν είναι όλα τα τμήματα του αλγορίθμου πλήρως vectorized.

5. Το speedup της SIMD έκδοσης περιορίστηκε από διάφορους παράγοντες. Πρώτον, ο αλγόριθμος επεξεργάζεται πολύ μεγάλο όγκο δεδομένων και απαιτεί δύο περάσματα από το αρχείο εισόδου. Στην πρώτη φάση διαβάζεται το αρχείο για τον υπολογισμό των στατιστικών, ενώ στη δεύτερη φάση διαβάζεται ξανά και γράφεται το κανονικοποιημένο αποτέλεσμα. Επομένως, μεγάλο μέρος του συνολικού χρόνου αφορά είσοδο/έξοδο και μεταφορά δεδομένων, όχι μόνο αριθμητικούς υπολογισμούς.

Δεύτερον, η πρόσβαση στη μνήμη παίζει σημαντικό ρόλο. Οι SIMD εντολές επιταχύνουν τις αριθμητικές πράξεις, αλλά αν ο επεξεργαστής περιμένει δεδομένα από τη μνήμη ή από τον δίσκο, η επιτάχυνση περιορίζεται. Σε αυτή την περίπτωση λέμε ότι ο αλγόριθμος είναι memory-bound ή I/O-bound.Τρίτον, δεν έγιναν πλήρως vectorized όλα τα τμήματα του αλγορίθμου. Στην πρώτη φάση, τα `sum` και `sumsq` υπολογίζονται με SIMD, όμως τα `min` και `max` υπολογίζονται με scalar τρόπο μέσα στο loop. Αυτό μειώνει το συνολικό όφελος από τη χρήση SIMD.
Τέταρτον, η πράξη της διαίρεσης που χρησιμοποιείται στο scaling είναι πιο αργή από απλές πράξεις όπως η πρόσθεση ή ο πολλαπλασιασμός. Επομένως, ακόμα και με SIMD, η δεύτερη φάση μπορεί να παραμένει σχετικά ακριβή.

Τέλος, υπάρχει και ένα μικρό overhead από τη διαχείριση των SIMD δεδομένων, όπως φορτώσεις, αποθηκεύσεις, scalar tail για τις περιπτώσεις όπου το D δεν είναι πολλαπλάσιο του 4, και έλεγχοι όπως η αποφυγή διαίρεσης με μηδέν στο minmax scaling.

```
xrist@MSI:~/parallel_scaler$ make run_simd_standard
./scaler_simd data_10M_32.bin out_simd.bin 10000000 32 standard

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (SIMD version with AVX2):
Phase 1 (Statistics): 756.96 ms
Phase 2 (Scaling):    2548.00 ms
Total:                3438.13 ms
```

Η SIMD έκδοση για standard scaling ολοκληρώθηκε σε 3438.13 ms. Η πρώτη φάση, δηλαδή ο υπολογισμός των στατιστικών, διήρκησε 756.96 ms, ενώ η δεύτερη φάση, δηλαδή η εφαρμογή του scaling και η εγγραφή του αποτελέσματος, διήρκησε 2548.00 ms.

Σε σχέση με τη σειριακή έκδοση, το speedup υπολογίζεται ως:
T_serial(Total) / T_SIMD(Total)=5259.03/3438.13=1.5296x. Εφόσον ο χρόνος της SIMD έκδοσης είναι μικρότερος από τον χρόνο της σειριακής, συμπεραίνουμε ότι η χρήση AVX2 SIMD βελτίωσε την απόδοση. Η βελτίωση οφείλεται στο ότι η SIMD έκδοση επεξεργάζεται 4 τιμές double ταυτόχρονα.

4.3. OpenMP

Στο ερώτημα 4.3 υλοποιήθηκε η παράλληλη έκδοση του scaler με χρήση OpenMP. Στόχος της υλοποίησης είναι η αξιοποίηση πολλών πυρήνων σε σύστημα κοινής μνήμης, ώστε να μειωθεί ο χρόνος εκτέλεσης σε σχέση με τη σειριακή έκδοση.

1. Τρόπος κατανομής εργασίας στα threads
Στην πρώτη φάση, η εργασία κατανέμεται στα threads με βάση τις γραμμές του τρέχοντος block. Για κάθε block δεδομένων, δημιουργείται parallel region με `#pragma omp parallel` και στη συνέχεια χρησιμοποιείται `#pragma omp for schedule(static)`. Έτσι, οι γραμμές του block μοιράζονται στα διαθέσιμα threads με στατικό τρόπο. Κάθε thread επεξεργάζεται ένα υποσύνολο γραμμών και για κάθε γραμμή διατρέχει όλες τις στήλες. Δηλαδή, ο παραλληλισμός γίνεται κυρίως ως προς τις γραμμές του πίνακα και όχι ως προς τις στήλες.
Στη δεύτερη φάση, όπου εφαρμόζεται το scaling, χρησιμοποιείται επίσης `#pragma omp parallel for schedule(static)`. Κάθε thread αναλαμβάνει διαφορετικές γραμμές του block και εφαρμόζει είτε StandardScaler είτε MinMaxScaler στα στοιχεία αυτών των γραμμών.

2. Στρατηγική αποφυγής race conditions
Στην πρώτη φάση υπάρχει κίνδυνος race condition, επειδή πολλά threads θα μπορούσαν να ενημερώνουν ταυτόχρονα τους ίδιους πίνακες `sum`, `sumsq`, `minv` και `maxv` για κάθε στήλη.
Για να αποφευχθεί αυτό, κάθε thread χρησιμοποιεί δικούς του τοπικούς πίνακες:

- thread_sum
- thread_sumsq
- thread_minv
- thread_maxv

Οι τοπικοί αυτοί πίνακες αποθηκεύουν τα μερικά αποτελέσματα κάθε thread. Έτσι, κατά την επεξεργασία των γραμμών, κάθε thread γράφει μόνο στη δική του μνήμη και δεν υπάρχει σύγκρουση με άλλα threads.
Μετά την ολοκλήρωση του τοπικού υπολογισμού, τα αποτελέσματα συγχωνεύονται στους συνολικούς πίνακες `sum`, `sumsq`, `minv` και `maxv` μέσα σε `#pragma omp critical`. Το critical section εξασφαλίζει ότι μόνο ένα thread κάθε φορά ενημερώνει τους global πίνακες, αποφεύγοντας race conditions.

Στη δεύτερη φάση δεν υπάρχει αντίστοιχος κίνδυνος race condition, επειδή κάθε thread γράφει σε διαφορετικές θέσεις του buffer. Οι πίνακες `mean`, `stddev`, `minv` και `maxv` χρησιμοποιούνται μόνο για ανάγνωση.

3. Χρήση reductions

Στην υλοποίηση δεν χρησιμοποιούνται OpenMP reduction clauses πάνω στους global πίνακες. Αντί για αυτό, εφαρμόζεται χειροκίνητη μορφή reduction. Κάθε thread υπολογίζει τοπικά αθροίσματα, αθροίσματα τετραγώνων, ελάχιστες και μέγιστες τιμές, και στο τέλος τα συγχωνεύει στους global πίνακες μέσα σε critical section.

Αυτή η στρατηγική είναι λειτουργικά ισοδύναμη με reduction, αλλά υλοποιείται χειροκίνητα επειδή τα στατιστικά είναι πίνακες ανά στήλη και περιλαμβάνουν διαφορετικές πράξεις: άθροισμα, άθροισμα τετραγώνων, ελάχιστο και μέγιστο.

```
hpcgrp02@krylov100:~/parallel_scaler$ make run_openmp_1
./scaler_openmp data_10M_32.bin out_openmp_1.bin 10000000 32 standard 1
OpenMP Configuration:
  Max threads available: 1
  Threads to use: 1
  Block size: 100000 rows

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (OpenMP version):
Phase 1 (Statistics): 2680.46 ms
Phase 2 (Scaling):    5159.80 ms
Total:                7840.45 ms
```

```
hpcgrp02@krylov100:~/parallel_scaler$ make run_openmp_2
./scaler_openmp data_10M_32.bin out_openmp_2.bin 10000000 32 standard 2
OpenMP Configuration:
  Max threads available: 2
  Threads to use: 2
  Block size: 100000 rows

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (OpenMP version):
Phase 1 (Statistics): 754.85 ms
Phase 2 (Scaling):    3007.04 ms
Total:               3762.05 ms
^[[A^[[Ahpcgrp02@krylov100:~/parallel_scaler$ make run_openmp_4
./scaler_openmp data_10M_32.bin out_openmp_4.bin 10000000 32 standard 4
OpenMP Configuration:
  Max threads available: 4
  Threads to use: 4
  Block size: 100000 rows

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (OpenMP version):
Phase 1 (Statistics): 671.29 ms
Phase 2 (Scaling):    3079.36 ms
Total:               3750.83 ms
```

```
hpcgrp02@krylov100:~/parallel_scaler$ make run_openmp_8
./scaler_openmp data_10M_32.bin out_openmp_8.bin 10000000 32 standard 8
OpenMP Configuration:
  Max threads available: 8
  Threads to use: 8
  Block size: 100000 rows

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (OpenMP version):
Phase 1 (Statistics): 671.68 ms
Phase 2 (Scaling):    2984.29 ms
Total:               3656.15 ms

hpcgrp02@krylov100:~/parallel_scaler$ make run_openmp_16
./scaler_openmp data_10M_32.bin out_openmp_16.bin 10000000 32 standard 16
OpenMP Configuration:
  Max threads available: 16
  Threads to use: 16
  Block size: 100000 rows

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (OpenMP version):
Phase 1 (Statistics): 767.75 ms
Phase 2 (Scaling):    3109.39 ms
Total:               3877.31 ms
```

Threads	Phase 1 Statistics	Phase 2 Scaling	Total
1	2680.46 ms	5159.80 ms	7840.45 ms
2	754.85 ms	3007.04 ms	3762.05 ms
4	671.29 ms	3079.36 ms	3750.83 ms
8	671.68 ms	2984.29 ms	3656.15 ms
16	767.75 ms	3109.39 ms	3877.31 ms

Speedup σε σχέση με 1 thread:

Threads	Total Time	Speedup
1	7840.45 ms	1.00x
2	3762.05 ms	2.08x
4	3750.83 ms	2.09x
8	3656.15 ms	2.14x
16	3877.31 ms	2.02x

Από τις μετρήσεις παρατηρούμε ότι η μετάβαση από 1 σε 2 threads μειώνει σημαντικά τον συνολικό χρόνο εκτέλεσης, από 7840.45 ms σε 3762.05 ms, δίνοντας speedup περίπου 2.08x. Ωστόσο, για περισσότερα threads η βελτίωση είναι μικρή. Οι χρόνοι για 4, 8 και 16 threads παραμένουν κοντά μεταξύ τους, με την καλύτερη μέτρηση να εμφανίζεται στα 8 threads, όπου το speedup είναι περίπου 2.14x.

Η συμπεριφορά αυτή δείχνει ότι η εφαρμογή δεν κλιμακώνει γραμμικά με τον αριθμό των threads. Παρόλο που οι υπολογισμοί των στατιστικών και του scaling παραλληλοποιούνται, μεγάλο μέρος του χρόνου επηρεάζεται από την ανάγνωση και εγγραφή μεγάλων binary αρχείων, καθώς και από το memory bandwidth. Επιπλέον, στην πρώτη φάση υπάρχει κόστος συγχώνευσης των τοπικών αποτελεσμάτων των threads μέσω critical section.

Συνεπώς, η OpenMP υλοποίηση βελτιώνει την απόδοση σε σχέση με την εκτέλεση με 1 thread, αλλά το speedup περιορίζεται μετά τα 2 threads λόγω I/O, πρόσβασης στη μνήμη και κόστους συγχρονισμού.

4.4 MPI Υλοποίηση

Η MPI έκδοση υλοποιήθηκε με στόχο την εκτέλεση του προγράμματος σε κατανεμημένη μνήμη, όπου κάθε διεργασία αναλαμβάνει ένα διαφορετικό τμήμα του πίνακα δεδομένων και εργάζεται ανεξάρτητα.

1. Κατανομή Γραμμών στις Διεργασίες

Η κατανομή των γραμμών υλοποιείται μέσω της συνάρτησης compute_range, η οποία μοιράζει τις N γραμμές ομοιόμορφα στις διαθέσιμες διεργασίες με διαχείριση του υπολοίπου:

```
long long base = N / size;

int rem = N % size;

if (rank < rem) {

   *local_N = base + 1;

   *start_row = rank * (*local_N);

} else {

   *local_N = base;

   *start_row = rem * (base + 1) + (rank - rem) * base;

}
```

Έτσι για N=10M και 3 διεργασίες, οι rank 0 και 1 παίρνουν ~3.34M γραμμές και ο rank 2 παίρνει ~3.33M. Κάθε διεργασία υπολογίζει μόνη της τα όρια της χωρίς επιπλέον επικοινωνία, κάτι που εξαλείφει ένα πιθανό σημείο συμφόρησης κατά την εκκίνηση.

2. Υπολογισμός Συνολικών Στατιστικών

Στην πρώτη φάση κάθε διεργασία διαβάζει το δικό της τμήμα και υπολογίζει τοπικά στατιστικά (local_sum, local_sumsq, local_min, local_max) χωρίς καμία επικοινωνία με τις υπόλοιπες. Μόλις ολοκληρωθεί η τοπική επεξεργασία, τα αποτελέσματα συγχωνεύονται μέσω MPI_Reduce:

```
MPI_Reduce(local_sum,   global_sum,   D, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);

MPI_Reduce(local_sumsq, global_sumsq, D, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);

MPI_Reduce(local_min,   global_min,   D, MPI_DOUBLE, MPI_MIN, 0,
MPI_COMM_WORLD);

MPI_Reduce(local_max,   global_max,   D, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);
```

Ο rank 0 υπολογίζει στη συνέχεια τον μέσο όρο, τη διακύμανση και την τυπική απόκλιση, και τα στέλνει σε όλες τις διεργασίες μέσω MPI_Bcast, ώστε η φάση scaling να εκτελεστεί παντού με τα ίδια στατιστικά.

3. MPI I/O αντί Κλασικού File I/O

Επιλέχθηκε MPI I/O αντί του κλασικού fopen/fread, ώστε κάθε διεργασία να διαβάζει και να γράφει παράλληλα το δικό της τμήμα του αρχείου χρησιμοποιώντας απόλυτα offsets:

MPI_Offset offset = ((MPI_Offset)(start_row + rows_done) * D) * sizeof(double);

MPI_File_read_at(infile, offset, buffer, elems, MPI_DOUBLE, &status);

Αυτό αποφεύγει το κλασικό πρόβλημα όπου μία μόνο διεργασία διαβάζει ολόκληρο το αρχείο και διανέμει τα δεδομένα στις υπόλοιπες, που θα δημιουργούσε σοβαρό bottleneck. Το ίδιο ισχύει και για την εγγραφή στη φάση 2, όπου κάθε διεργασία γράφει το scaled τμήμα της στη σωστή θέση του αρχείου εξόδου.

4. Κόστος Συλλογικών Επικοινωνιών

Το MPI_Reduce χρησιμοποιεί tree-based αλγόριθμο με κόστος O(log P), αντί O(P) που θα είχε μία naive προσέγγιση. Για P=16 διεργασίες αυτό σημαίνει ~4 βήματα επικοινωνίας για το Reduce και άλλα ~4 για το Broadcast — συνολικά αμελητέο κόστος σε σχέση με τον χρόνο I/O, γεγονός που επιβεβαιώνεται από τις μετρήσεις όπου η Phase 1b (compute + bcast) διαρκεί μόλις 0.30-0.31 ms.

5. Bottleneck στο Filesystem

Με MPI I/O δεν παρατηρήθηκε σημαντικό filesystem bottleneck στις φάσεις ανάγνωσης και εγγραφής, αφού όλες οι διεργασίες προσπελαύνουν διαφορετικά τμήματα του αρχείου παράλληλα. Η μόνη πτώση απόδοσης εμφανίζεται στις 16 διεργασίες (2397 ms Phase 2 έναντι 2035 ms στις 8), πιθανώς λόγω contention στον κοινόχρηστο δίσκο του Krylov100 από άλλους χρήστες. Αν είχε χρησιμοποιηθεί κλασικό fopen/fread με serial ανάγνωση από έναν κόμβο, το bottleneck θα ήταν εμφανές από τις πρώτες κιόλας μετρήσεις.

Screenshots εκτέλεσης για διαφορετικό αριθμό των threads:

```
mpirun -np 1 ./scaler_mpi data_10M_32.bin out_mpi_1.bin 10000000 32 standard

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (ms):
Phase 1 (local stats + reduce): 863.80 ms
Phase 1b (compute+bcast):     0.00 ms
Phase 2 (scaling+IO):         12009.38 ms
Total:                        12873.18 ms
```

```
mpirun -np 4 ./scaler_mpi data_10M_32.bin out_mpi_4.bin 10000000 32 standard

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (ms):
Phase 1 (local stats + reduce): 267.62 ms
Phase 1b (compute+bcast):     0.02 ms
Phase 2 (scaling+IO):         3339.80 ms
Total:                        3607.45 ms
```

```
mpirun -np 2 ./scaler_mpi data_10M_32.bin out_mpi_2.bin 10000000 32 standard

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (ms):
Phase 1 (local stats + reduce): 504.15 ms
Phase 1b (compute+bcast):     0.02 ms
Phase 2 (scaling+IO):         7684.58 ms
Total:                        8188.75 ms
```

```
mpirun -np 8 ./scaler_mpi data_10M_32.bin out_mpi_8.bin 10000000 32 standard

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (ms):
Phase 1 (local stats + reduce): 246.35 ms
Phase 1b (compute+bcast):     0.31 ms
Phase 2 (scaling+IO):         2035.15 ms
Total:                        2281.81 ms
```

```
mpirun -np 16 ./scaler_mpi data_10M_32.bin out_mpi_16.bin 10000000 32 standard

Sample statistics:
Column 0
  Mean     = 0.000108624
  Stddev   = 0.999912
  Min      = -5.21936
  Max      = 5.26056
Column 1
  Mean     = 4.6554e-05
  Stddev   = 0.999888
  Min      = -5.09636
  Max      = 5.11765
Column 2
  Mean     = 6.11601e-05
  Stddev   = 0.99982
  Min      = -5.64832
  Max      = 5.46937
Column 3
  Mean     = 0.000317352
  Stddev   = 0.999772
  Min      = -5.12469
  Max      = 5.61194
Column 4
  Mean     = -0.000183954
  Stddev   = 0.999989
  Min      = -5.20209
  Max      = 5.43043
Scaling completed successfully.

Timing Information (ms):
Phase 1 (local stats + reduce): 157.58 ms
Phase 1b (compute+bcast):     0.30 ms
Phase 2 (scaling+IO):         2397.42 ms
Total:                        2555.30 ms
```

MPI Processes	Phase 1 local stats + reduce	Phase 1b compute + bcast	Phase 2 scaling + I/O	Total
1	863.80 ms	0.00 ms	12009.38 ms	12873.18 ms
2	504.15 ms	0.02 ms	7684.58 ms	8188.75 ms
4	267.62 ms	0.02 ms	3339.80 ms	3607.45 ms
8	246.35 ms	0.31 ms	2035.15 ms	2281.81 ms
16	157.58 ms	0.30 ms	2397.42 ms	2555.30 ms

MPI Processes	Total Time	Speedup
1	12873.18 ms	1.00x
2	8188.75 ms	1.57x
4	3607.45 ms	3.57x
8	2281.81 ms	5.64x
16	2555.30 ms	5.04x

Συμπέρασμα:

Από τις μετρήσεις παρατηρούμε ότι η MPI υλοποίηση βελτιώνει σημαντικά την απόδοση όσο αυξάνεται ο αριθμός των διεργασιών μέχρι τις 8 διεργασίες. Ο συνολικός χρόνος μειώνεται από 12873.18 ms με 1 διεργασία σε 2281.81 ms με 8 διεργασίες, επιτυγχάνοντας speedup περίπου 5.64x.Στις 16 διεργασίες ο χρόνος αυξάνεται σε 2555.30 ms και το speedup μειώνεται σε 5.04x. Αυτό δείχνει ότι μετά από ένα σημείο η αύξηση των MPI διεργασιών δεν οδηγεί σε περαιτέρω βελτίωση. Η πιθανότερη αιτία είναι το αυξημένο κόστος επικοινωνίας και η συμφόρηση στο filesystem

κατά τη φάση του scaling και της εγγραφής του αποτελέσματος.Η Phase 1 μειώνεται σταθερά όσο αυξάνονται οι διεργασίες, επειδή κάθε διεργασία υπολογίζει στατιστικά σε μικρότερο τμήμα των δεδομένων. Αντίθετα, η Phase 2 παραμένει το πιο ακριβό τμήμα της εκτέλεσης, καθώς περιλαμβάνει τόσο την εφαρμογή του scaling όσο και την είσοδο/έξοδο στο αρχείο.Τελικά,η καλύτερη επίδοση εμφανίζεται με 8 MPI διεργασίες, όπου επιτυγχάνεται speedup 5.64x. Με 16 διεργασίες η απόδοση μειώνεται ελαφρώς, πιθανώς λόγω I/O bottleneck και αυξημένου κόστους συντονισμού/επικοινωνίας.

4.5 CUDA Υλοποίηση

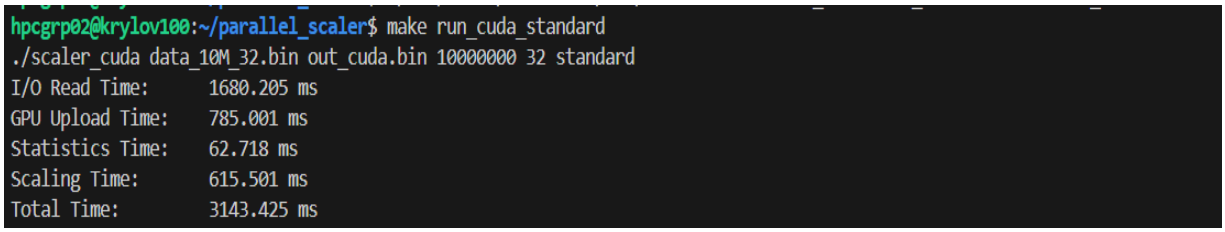
Για την υλοποίηση και εκτέλεση της CUDA έκδοσης χρησιμοποιήθηκε ο διακομιστής **Krylov100** καθώς το τοπικό μηχάνημα δεν διαθέτει την απαραίτητη GPU. Η πρόσβαση έγινε μέσω SSH:

ssh hpcgrp02@scgroup3.ceid.upatras.gr

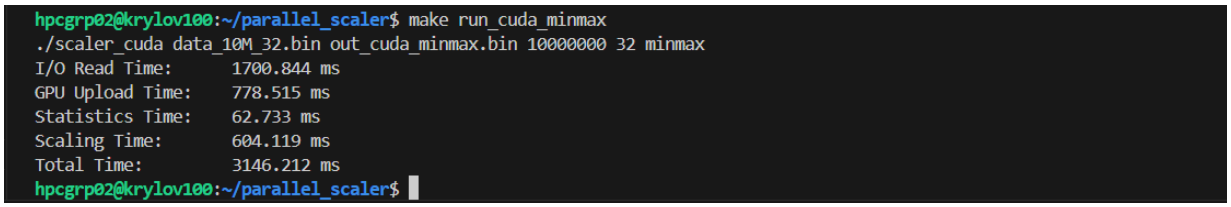
Στη συγκεκριμένη υλοποίηση, η CPU αναλαμβάνει την ανάγνωση του raw binary αρχείου από τον δίσκο, τη δέσμευση μνήμης στη CPU και στη GPU, τη μεταφορά των δεδομένων στη GPU, τη μεταφορά των αποτελεσμάτων πίσω στη CPU και την εγγραφή του τελικού raw binary αρχείου εξόδου. Η GPU αναλαμβάνει τον υπολογισμό των στατιστικών και την εφαρμογή του scaling.Για τον υπολογισμό των στατιστικών χρησιμοποιούνται CUDA kernels που υπολογίζουν μερικά αποτελέσματα για κάθε στήλη. Συγκεκριμένα, το `compute_sum_sumsq_kernel` υπολογίζει το άθροισμα και το άθροισμα τετραγώνων, ενώ το `compute_minmax_kernel` υπολογίζει την ελάχιστη και τη μέγιστη τιμή. Οι kernels χρησιμοποιούν shared memory για reduction μέσα σε κάθε CUDA block. Τα μερικά αποτελέσματα των blocks αντιγράφονται πίσω στη CPU, όπου γίνεται η τελική συγχώνευση για κάθε στήλη.

Μετά τον υπολογισμό των στατιστικών, οι πίνακες `mean`, `stddev`, `minv` και `maxv` αντιγράφονται στη GPU. Στη συνέχεια εφαρμόζεται το scaling με διαφορετικό kernel ανάλογα με το mode.Η CUDA υλοποίηση μετρά ξεχωριστά τον χρόνο ανάγνωσης από τον δίσκο, τον χρόνο μεταφοράς των δεδομένων στη GPU, τον χρόνο υπολογισμού στατιστικών, τον χρόνο εφαρμογής του scaling και τον συνολικό χρόνο. Αυτό επιτρέπει να φανεί αν το κόστος βρίσκεται στους GPU kernels ή στις μεταφορές δεδομένων και στο I/O.

Screenshot εκτέλεσης για standard scaling:



Screenshot εκτέλεσης για min-max scaling:



Από τις μετρήσεις παρατηρούμε ότι οι καθαροί υπολογισμοί στη GPU είναι εξαιρετικά γρήγοροι. Ωστόσο, ο συνολικός χρόνος κυριαρχείται από το I/O read time, δηλαδή από την ανάγνωση του μεγάλου binary αρχείου από τον δίσκο. Για το standard scaling, η ανάγνωση απαιτεί περίπου **1.68 δευτερόλεπτα** από συνολικά **3.14 δευτερόλεπτα** (~53% του συνολικού χρόνου). Για το minmax scaling, η ανάγνωση απαιτεί περίπου **1.70 δευτερόλεπτα** από συνολικά **3.15 δευτερόλεπτα**.

Η συγκεκριμένη έκδοση δεν υλοποιεί πλήρως block-by-block επεξεργασία στη GPU. Αφού το dataset δεν χωράει ολόκληρο στη μνήμη της GPU, η δέσμευση μνήμης με cudaMalloc ή η μεταφορά με cudaMemcpy μπορεί να αποτύχει. Για να υποστηριχθούν μεγαλύτερα datasets, θα έπρεπε η εφαρμογή να διαβάζει τμήματα του αρχείου, να τα μεταφέρει σταδιακά στη GPU, να υπολογίζει μερικά στατιστικά ανά block και να τα συγχωνεύει. Στη συνέχεια θα απαιτούνταν δεύτερο πέρασμα στα blocks για την εφαρμογή του scaling και την εγγραφή του αποτελέσματος.

3.Συνοπτικά αποτελέσματα

Έκδοση	Phase 1 (ms)	Phase 2 (ms)	Σύνολο (ms)	Speedup vs Serial
Serial	—	—	5259.03	1.00x
SIMD (AVX2)	756.96	2548.00	3438.13	1.53x
OpenMP (1 thread)	2680.46 ms	5159.80 ms	7840.45 ms	0.67x
OpenMP (2 threads)	754.85 ms	3007.04 ms	3762.05 ms	1.40x
OpenMP (4 threads)	671.29 ms	3079.36 ms	3750.83 ms	1.40x
OpenMP (8 threads)	671.68 ms	2984.29 ms	3656.15 ms	1.44x
OpenMP (16 threads)	767.75 ms	3109.39 ms	3877.31 ms	1.36x
MPI (1 threads)	863.80 ms	0.00 ms	12009.38 ms	0.44x
MPI (2 threads)	504.15 ms	0.02 ms	7684.58 ms	0.68x
MPI Processes(4 threads)	267.62 ms	0.02 ms	3339.80 ms	1.57x
MPI Processes(8 threads)	246.35 ms	0.31 ms	2035.15 ms	2.58x
MPI Processes(16 threads)	157.58 ms	0.30 ms	2397.42 ms	2.19x

Μέτρηση	Standard Scaling	MinMax Scaling
I/O Read Time	1680.205 ms	1700.844 ms
GPU Upload Time	785.001 ms	778.515 ms
Statistics Time	62.718 ms	62.733 ms
Scaling Time	615.501 ms	604.119 ms
Total Time	3143.425 ms	3146.212 ms
Speedup vs Serial	1.67x	1.67x

4. Συμπεράσματα

Ο αλγόριθμος είναι κυρίως I/O-bound. Το μεγαλύτερο μέρος του συνολικού χρόνου εκτέλεσης δεν οφείλεται στους αριθμητικούς υπολογισμούς αλλά στην ανάγνωση και εγγραφή δυαδικών αρχείων μεγάλου μεγέθους. Αυτό επιβεβαιώνεται από το γεγονός ότι η Phase 2 (scaling + εγγραφή) είναι σε όλες τις εκδόσεις σαφώς πιο χρονοβόρα από την Phase 1 (υπολογισμός στατιστικών). Κατά συνέπεια, ο παραλληλισμός των αριθμητικών πράξεων από μόνος του δεν αρκεί για υψηλό speedup.

Η SIMD/AVX2 υλοποίηση έχει speedup **1.53x** έναντι της σειριακής, επεξεργαζόμενη 4 τιμές double ταυτόχρονα με explicit AVX2 intrinsics. Ήταν η πιο αποδοτική single-thread λύση, καθώς δεν εισάγει overhead συγχρονισμού ή επικοινωνίας

Η OpenMP υλοποίηση κλιμακώνει μόνο μέχρι τα 2 threads (speedup **1.40x**) και στη συνέχεια παραμένει σχεδόν σταθερή, με το βέλτιστο να εμφανίζεται στα 8 threads (**1.44x**). Στα 16 threads έχουμε πτώση του speedup λόγω αυξημένου overhead στα critical sections. Η Phase 2 δεν επιταχύνεται σημαντικά με περισσότερα threads επειδή το bottleneck είναι το I/O και το memory bandwidth. Αξιοσημείωτο είναι ότι το OpenMP με 1 thread είναι **πιο αργό** από τη σειριακή (0.67x) λόγω overhead δημιουργίας parallel region και critical sections χωρίς παράλληλη εκτέλεση.

Η MPI υλοποίηση έδειξε την καλύτερη κλιμάκωση από όλες τις εκδόσεις, με speedup **2.58x** στις 8 διεργασίες. Αυτό οφείλεται στο ότι κάθε διεργασία διαβάζει παράλληλα το δικό της τμήμα του αρχείου μέσω MPI I/O, αξιοποιώντας αποδοτικά το διαθέσιμο bandwidth. Με 1 και 2 διεργασίες το MPI είναι πιο αργό από τη σειριακή (0.44x και 0.68x αντίστοιχα) λόγω overhead εκκίνησης.

Η **CUDA** υλοποίηση παρουσίασε **speedup** 1.67x, με συνολικό χρόνο 3143.425 ms για standard και 3146.212 ms για minmax, δηλαδή **σημαντικά γρηγορότερη** από τη σειριακή (5259 ms). Παρατηρούμαι ότι οι καθαροί GPU υπολογισμοί (στατιστικά + scaling) ολοκληρώνονται σε μόλις ~678 ms,ένω ο **υπόλοιπος χρόνου αφορά I/O και μεταφορά δεδομένων** CPU↔GPU. Αυτό σημαίνει ότι η GPU αξιοποιείται πλήρως για τους αριθμητικούς υπολογισμούς, και το speedup περιορίζεται αποκλειστικά από εξωτερικούς παράγοντες που δεν σχετίζονται με την υπολογιστική ισχύ της.